

Département de Mathématiques et Informatique

Projet de fin de Module

MLOps & DataOps

EduScore MLOps

**Plateforme DataOps et MLOps pour la prédiction du
score final des étudiants**

Réalisé par :

MOHAMMED HMIMID

Aldiebes Ghanem ISRAA

Chakir Abderrahmane

Khadija Zaafa

BAHRI ILHAME

Othman SALAHI

Aymene Chouki

Makrani Mohamed

Demandé par :

Pr. Mohammed AIT DAOUD

Année universitaire : 2025 / 2026

Table des matières

Résumé	6
1. Introduction générale.....	7
1.1 Contexte	7
1.2 Objectif du projet	7
2. Vision métier et stratégie data	7
2.1 Problématique.....	7
2.2 Utilisateurs et valeur métier	8
2.3 Data Product et Data Consumers	8
3. Organisation Agile, Jira et GitHub.....	8
3.1 Démarche Agile.....	8
3.2 Répartition des tâches.....	9
3.3 Collaboration GitHub	9
4. Architecture globale de la solution.....	10
5. Pipeline DataOps suivi	11
5.1 Restauration et versionnement de la donnée	11
5.2 Ingestion avec dlt	11
5.3 Transformations avec dbt	11
5.4 Dataset fingerprint.....	12
6. Orchestration Dagster.....	13
7. Qualité des données et tests.....	14
7.1 Data contract	14
7.2 Tests automatisés.....	15
8. Machine Learning	16
8.1 Préparation des données	16
8.2 Modèle entraîné.....	16
8.3 Résultats obtenus	16
9. Mlflow et MLOps.....	17
10. Déploiement et API.....	18

10.1 FastAPI.....	18
10.2 Docker	19
11. CI/CD et collaboration technique.....	19
12. Monitoring et observabilité	20
13. Validation globale et résultats de démonstration	20
14. Synthèse des livrables réalisés	21
Conclusion.....	22
Bibliographie	23
Annexe A: Commandes complètes d'exécution	24
Annexe B: Exemple de requête de prédiction	24

Liste des tableaux

Tableau 1 : Stratégie data appliquée	8
Tableau 2 : Synthèse Jira.....	9
Tableau 3 : Répartition fonctionnelle des tâches	9
Tableau 4 : Composants de l'architecture.....	10
Tableau 5 : Artefacts DataOps générés	12
Tableau 6 : Assets Dagster réalisés	13
Tableau 7 : Schéma canonique du data contract	14
Tableau 8 : Tests réalisés	15
Tableau 9 : Résultats du modèle	16
Tableau 10 : Informations suivies dans Mlflow	17
Tableau 11 : Endpoints FastAPI.....	18
Tableau 12 : Pipeline CI/CD GitHub Actions.....	19
Tableau 13 : Données observées en monitoring.....	20
Tableau 14 : Checklist de validation finale	21
Tableau 15 : Correspondance entre exigences du module et réalisations	21

Liste des figures

Figure 1 : Schéma global DataOps et MLOps	11
Figure 2 : Schéma d'orchestration Dagster.....	13
Figure 3 : Matérialisation réussie dans Dagster	14
Figure 4 : Pipeline de Tests Automatisés	15
Figure 5 : Capture des résultats des tests.....	15
Figure 6 : Capture de l'entraînement du modèle.....	16
Figure 7 : Capture MLflow des runs	17
Figure 8 : Capture MLflow du modèle enregistré.....	18
Figure 9 : Pipeline CI/CD.....	20

Résumé

Ce rapport présente le projet EduScore MLOps, réalisé dans le cadre du module MLOps & DataOps. Le projet consiste à concevoir et industrialiser une solution complète de Data et Intelligence Artificielle permettant de prédire le score final d'un étudiant à partir de signaux académiques simples, notamment les heures d'étude hebdomadaires, le taux de présence et la participation en classe. L'objectif principal n'est donc pas seulement de développer un modèle de Machine Learning, mais de construire un véritable produit data exploitable, reproductible et maintenable.

La solution réalisée couvre l'ensemble du cycle de vie d'un projet Data/IA: définition de la stratégie data, organisation Agile avec Jira, collaboration GitHub, versionnement des données avec DVC, ingestion automatisée avec dlt, stockage analytique avec DuckDB, transformations avec dbt, orchestration avec Dagster, tests qualité, entraînement du modèle avec scikit-learn, suivi des expériences avec MLflow, exposition du modèle via FastAPI, conteneurisation avec Docker, intégration continue avec GitHub Actions et monitoring des prédictions.

Le rapport est structuré selon la démarche réellement suivie pendant le projet. Il présente les concepts mobilisés, l'architecture mise en place, le pipeline DataOps/MLOps construit, les résultats obtenus, les mécanismes de validation, ainsi que les preuves attendues sous forme de captures d'écran de Jira, GitHub, Dagster, MLflow, FastAPI, Docker et des résultats de commandes.

1. Introduction générale

1.1 Contexte

Les projets d'intelligence artificielle modernes ne se limitent plus à la construction d'un modèle prédictif isolé. Dans un contexte professionnel, un modèle doit être intégré dans une chaîne complète capable de gérer les données, d'assurer leur qualité, de suivre les expériences, de sauvegarder les artefacts, de déployer le service et de surveiller son comportement après mise en production. Sans ces éléments, même un modèle performant reste difficile à reproduire, à maintenir et à exploiter.

Les pratiques DataOps et MLOps répondent précisément à ces enjeux. Le DataOps permet d'automatiser et de fiabiliser le cycle de vie des données, depuis leur ingestion jusqu'à leur transformation et validation. Le MLOps permet d'industrialiser le cycle de vie du Machine Learning, depuis l'entraînement jusqu'au suivi des performances, au déploiement et au monitoring. Le projet EduScore MLOps applique ces principes dans un cas d'usage éducatif concret.

1.2 Objectif du projet

L'objectif du projet est de transformer un dataset brut de performance étudiante en un service ML exploitable, maintenable et démontrable. À partir de données simples comme les heures d'étude, la présence et la participation, le système produit une prédiction du score final ainsi qu'un grade interprétable. Cette prédiction peut servir d'indicateur précoce pour identifier les étudiants nécessitant un accompagnement pédagogique.

La valeur du projet réside autant dans le modèle que dans la chaîne complète permettant de le produire, le suivre, le servir et le contrôler. Le projet met donc en œuvre une architecture complète incluant l'ingestion des données, leur transformation, les tests qualité, l'entraînement du modèle, le suivi MLflow, l'orchestration Dagster, l'API FastAPI, Docker, GitHub Actions et un monitoring simple des prédictions.

2. Vision métier et stratégie data

2.1 Problématique

Les établissements d'enseignement ont besoin d'indicateurs précoces pour détecter les étudiants qui risquent d'obtenir une faible performance finale. EduScore MLOps propose une estimation du score final à partir de variables faciles à collecter.

2.2 Utilisateurs et valeur métier

- Enseignants: identifier les étudiants nécessitant un accompagnement.
- Coordinateurs pédagogiques: suivre la performance globale d'une cohorte.
- Équipes Data/ML: maintenir un pipeline reproductible et observable.
- Évaluateurs du projet: vérifier l'application des exigences MLOps et DataOps.

2.3 Data Product et Data Consumers

Le produit data final est un service de prédiction consommable par API. Les consommateurs sont l'API FastAPI, le pipeline d'entraînement, les outils de monitoring, MLflow et les utilisateurs métier qui interprètent le score et le grade.

Couche	Réalisation
Source	Dataset Student Performance issu de Kaggle.
Versionnement	DVC avec remote Google Drive et pointeur data/raw/student_performance.csv.dvc.
Ingestion	Pipeline dlt vers DuckDB.
Stockage	Base analytique locale data/student_score.duckdb.
Transformation	Modèles dbt: staging et dataset ML.
Qualité	Data contract, tests dbt et tests pytest.
ML	Régression supervisée avec scikit-learn.
Tracking	MLflow: paramètres, métriques, artefacts, registry.
Serving	FastAPI: /health, /model-info, /predict.
Monitoring	Logs JSONL, latence, métriques modèle et fingerprint dataset.

Tableau 1 : Stratégie data appliquée

3. Organisation Agile, Jira et GitHub

3.1 Démarche Agile

Le projet a été organisé avec Jira sous le nom SmartScoreAI. Le backlog contient des epics, user stories et subtasks couvrant la compréhension des données, le nettoyage, la modélisation, le tracking, le déploiement, le CI/CD, le monitoring et la documentation.

Élément	Résultat
Nombre total de tickets	36
Epics	6
Stories	18
Subtasks	12
Axes de sprint	Setup & Data Understanding, Modélisation ML, MLOps, CI/CD & Docker, Monitoring & Documentation.

Tableau 2 : Synthèse Jira

3.2 Répartition des tâches

Membre	Responsabilités principales
MOHAMMED HMIMID	Initialisation Git, configuration, exploration, Dockerisation, déploiement API.
Chakir Abderrahmane	Évaluation, MLflow, pipeline automatisé, versioning, pilotage Jira.
Makrani Mohamed	Nettoyage des données, CI/CD GitHub Actions, contribution GitHub.
aymene chouki	Variable cible, monitoring simple, analyse de performance.
Aldiebes Ghanem ISRAA	Modèle baseline, tests unitaires, nettoyage et prétraitement.
Khadija Zaafa	Comparaison des modèles, documentation, monitoring.
Othman SALAHI	Séparation train/test et contribution MLOps.
BAHRI ILHAME	Documentation technique, validation du pipeline DataOps, tests de démonstration.

Tableau 3 : Répartition fonctionnelle des tâches

3.3 Collaboration GitHub

Le dépôt GitHub centralise le code source, la documentation, les tests, la configuration Docker, le workflow CI/CD et les fichiers de configuration DVC. Les artefacts générés tels que les modèles, les bases DuckDB, les logs et les runs MLflow ne sont pas destinés à être versionnés dans Git.

4. Architecture globale de la solution

L'architecture suit une chaîne de bout en bout allant de la donnée brute au service de prédiction. Chaque composant répond à une exigence du module et joue un rôle précis dans l'industrialisation du produit IA.

Composant	Outil	Rôle
Versionnement données	DVC / Google Drive	Restaurer et partager la version exacte du dataset.
Ingestion	dlt	Lire le CSV et charger les données dans DuckDB.
Stockage	DuckDB	Stockage analytique local.
Transformation	dbt	Nettoyer, typer et produire le dataset ML.
Orchestration	Dagster	Orchestrer le pipeline complet et planifier les runs.
Qualité	dbt + pytest + Pydantic	Contrôler données, schémas et API.
Modèle	scikit-learn	Prédire le score final.
Tracking	MLflow	Suivre expériences, métriques, artefacts et registry.
Service	FastAPI	Exposer le modèle à travers une API.
Déploiement	Docker	Exécuter l'API et MLflow dans des conteneurs.
CI/CD	GitHub Actions	Automatiser lint, tests et build Docker.
Monitoring	JSONL logs	Tracer prédictions, grades et latence.

Tableau 4 : Composants de l'architecture

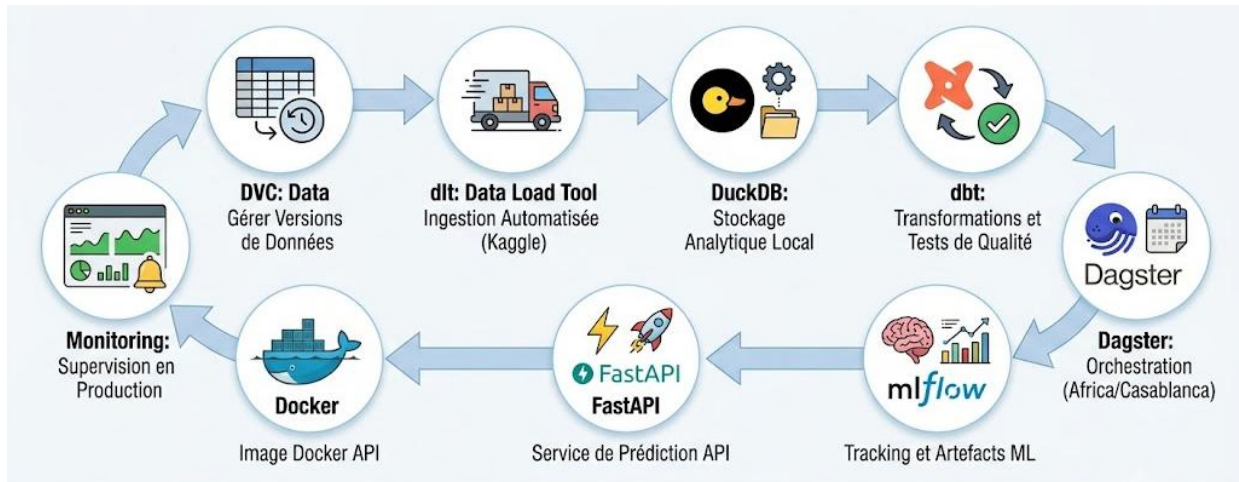


Figure 1 : Schéma global DataOps et MLOps

5. Pipeline DataOps suivi

5.1 Restauration et versionnement de la donnée

Le dataset brut est restauré avec DVC. Le fichier CSV réel n'est pas commité dans Git; seul le pointeur DVC est versionné. Cette pratique garantit un dépôt léger et une donnée reproductible.

```
scripts\dvc.cmd pull
scripts\dvc.cmd status
```

5.2 Ingestion avec dlt

Le script d'ingestion lit data/raw/student_performance.csv, standardise les colonnes et charge les données dans DuckDB sous une table brute.

```
(.venv) PS C:\Users\m\Desktop\Master IA ben msik\S2\MLOps\Projet> python dlt_pipeline/load_student_data.py
Loaded 1000000 rows into C:/Users/m/Desktop/Master IA ben msik/S2/MLOps/Projet/data/student_score.duckdb::raw.student_performance
```

5.3 Transformations avec dbt

dbt construit une couche staging puis un dataset final ml_student_training_set. Cette table contient les variables nécessaires au modèle et applique les règles de validité.

```

(.venv) PS C:\Users\m\Desktop\Master IA ben msik\S2\MLOps\Projet> dbt run --project-dir dbt/student_score --profiles-dir dbt/stu
ent_score
19:47:36 Running with dbt=1.8.4
19:47:38 Registered adapter: duckdb=1.8.2
19:47:44 Found 2 models, 7 data tests, 421 macros
19:47:44 Concurrency: 4 threads (target='dev')
19:47:53 1 of 2 START sql view model main.stg_student_performance ..... [RUN]
19:47:54 1 of 2 OK created sql view model main.stg_student_performance ..... [OK in 0.40s]
19:47:54 2 of 2 START sql table model main.ml_student_training_set ..... [RUN]
19:47:54 2 of 2 OK created sql table model main.ml_student_training_set ..... [OK in 0.65s]
19:47:54 Finished running 1 view model, 1 table model in 0 hours 0 minutes and 10.13 seconds (10.13s).
19:47:55 Completed successfully
19:47:55 Done. PASS=2 WARN=0 ERROR=0 SKIP=0 TOTAL=2

(.venv) PS C:\Users\m\Desktop\Master IA ben msik\S2\MLOps\Projet> dbt test --project-dir dbt/student_score --profiles-dir dbt/stu
dent_score
20:30:00 Running with dbt=1.8.4
20:30:01 Registered adapter: duckdb=1.8.2
20:30:05 Found 2 models, 7 data tests, 421 macros
20:30:09 Concurrency: 4 threads (target='dev')
20:30:09 1 of 7 START test accepted_values_ml_student_training_set_derived_grade_A_B_C_D_F [RUN]
20:30:09 2 of 7 START test accepted_values_stg_student_performance_grade_A_B_C_D_F . [RUN]
20:30:09 3 of 7 START test not_null_ml_student_training_set_total_score ..... [RUN]
20:30:09 4 of 7 START test not_null_stg_student_performance_attendance_percentage ..... [RUN]
20:30:09 3 of 7 PASS not_null_ml_student_training_set_total_score ..... [PASS in 0.16s]
20:30:09 4 of 7 PASS not_null_stg_student_performance_attendance_percentage ..... [PASS in 0.16s]
20:30:09 5 of 7 START test not_null_stg_student_performance_class_participation ..... [RUN]
20:30:09 6 of 7 START test not_null_stg_student_performance_study_hours ..... [RUN]
20:30:09 2 of 7 PASS accepted_values_stg_student_performance_grade_A_B_C_D_F ..... [PASS in 0.20s]
20:30:09 1 of 7 PASS accepted_values_ml_student_training_set_derived_grade_A_B_C_D_F [PASS in 0.21s]
20:30:09 7 of 7 START test not_null_stg_student_performance_total_score ..... [RUN]
20:30:09 5 of 7 PASS not_null_stg_student_performance_class_participation ..... [PASS in 0.11s]
20:30:09 6 of 7 PASS not_null_stg_student_performance_study_hours ..... [PASS in 0.10s]
20:30:09 7 of 7 PASS not_null_stg_student_performance_total_score ..... [PASS in 0.06s]
20:30:09 Finished running 7 data tests in 0 hours 0 minutes and 4.35 seconds (4.35s).
20:30:10 Completed successfully
20:30:10 Done. PASS=7 WARN=0 ERROR=0 SKIP=0 TOTAL=7

```

5.4 Dataset fingerprint

Après transformation, le dataset d'entraînement reçoit un fingerprint déterministe. Ce fingerprint permet de savoir si les données ont changé et d'éviter des réentraînements inutiles.

Artefact	Description
data/student_score.duckdb	Base DuckDB générée localement.
metadata/data_fingerprint.json	Fingerprint du dataset d'entraînement.
models/training_reference.json	Moyennes de référence des features et nombre de lignes.
dbt/student_score/target	Artefacts dbt générés localement.

Tableau 5 : Artefacts DataOps générés

6. Orchestration Dagster

Dagster représente la couche d'orchestration. Il enchaîne les assets de contrôle, ingestion, transformation, validation, fingerprint et entraînement. Une planification quotidienne est définie à 02:00 dans le fuseau Africa/Casablanca.

Asset	Rôle
check_raw_data_freshness	Vérifie l'existence et l'état de la donnée brute.
ingest_raw_student_data	Exécute l'ingestion dlt.
transform_student_data	Exécute dbt run.
validate_student_data	Exécute dbt test.
validate_python_contracts	Exécute pytest -q.
training_dataset_fingerprint	Calcule le fingerprint du dataset ML.
train_student_score_model	Entraîne si les données ont changé ou si force_train est activé.

Tableau 6 : Assets Dagster réalisés

Tableau 7 - Assets Dagster réalisés

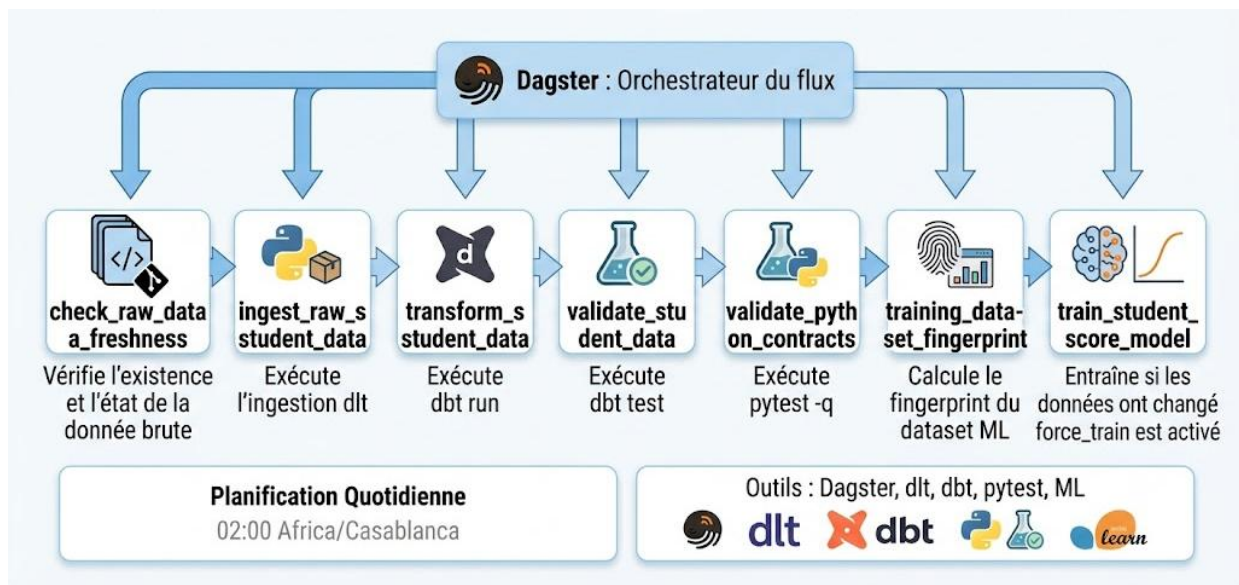


Figure 2 : Schéma d'orchestration Dagster

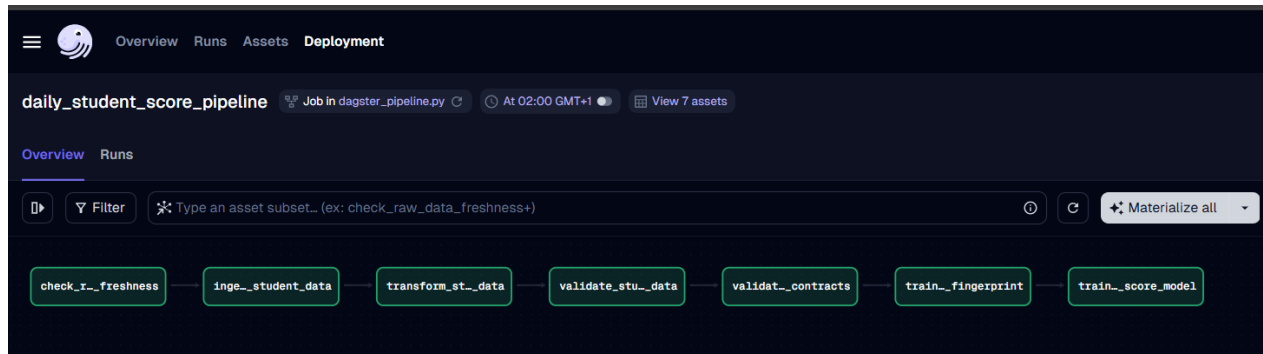


Figure 3 : Matérialisation réussie dans Dagster

7. Qualité des données et tests

7.1 Data contract

Colonne	Type	Règle
study_hours	float	$0 \leq \text{valeur} \leq 40$
attendance_percentage	float	$0 \leq \text{valeur} \leq 100$
class_participation	float	$0 \leq \text{valeur} \leq 10$
total_score	float	$0 \leq \text{valeur} \leq 100$
grade	string	A, B, C, D ou F

Tableau 7 : Schéma canonique du data contract

The screenshot shows the Dagster deployment interface for the `dagster_pipeline.py` file. The interface displays a list of operations (Ops) with their input and output types. The operations are:

- `check_raw_data_freshness`: `() => (result: Dict[String,Any])`
- `ingest_raw_student_data`: `(check_raw_data_freshness: Nothing) => (result: String)`
- `train_student_score_model`: `(training_dataset_fingerprint: Dict[String,Any]) => (result: Dict[String,Any])`
- `training_dataset_fingerprint`: `(validate_python_contracts: Nothing) => (result: Dict[String,Any])`
- `transform_student_data`: `(ingest_raw_student_data: Nothing) => (result: String)`
- `validate_python_contracts`: `(validate_student_data: Nothing) => (result: String)`
- `validate_student_data`: `(transform_student_data: Nothing) => (result: String)`

7.2 Tests automatisés

Type de test	Outil	Objectif
Tests data contract	pytest	Valider colonnes, bornes et règles métier.
Tests schéma API	Pydantic / pytest	Rejeter les entrées invalides.
Tests data versioning	pytest	Valider fingerprint et métadonnées.
Tests model-info	pytest	Valider la lecture des métriques et du fingerprint.
Tests dbt	dbt test	Valider not_null et accepted_values.

Tableau 8 : Tests réalisés

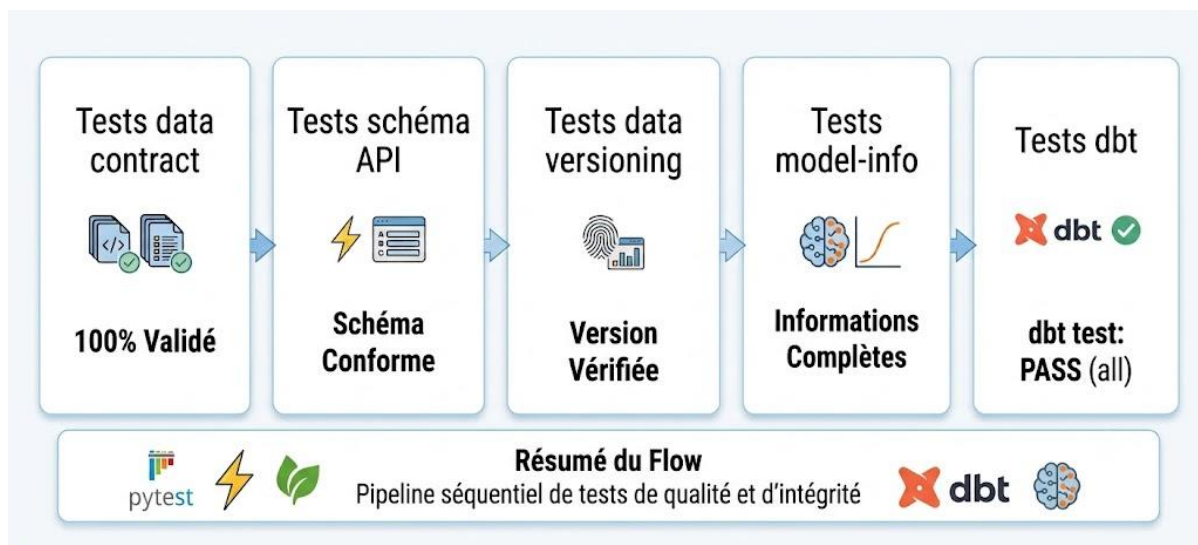


Figure 4 : Pipeline de Tests Automatisés

Résultat observé pour la suite Python: 12 tests réussis.

```

• (.venv) PS C:\Users\m\Desktop\Master IA ben msik\S2\MLOps\Projet> pytest -q
.....
12 passed in 2.26s

```

Figure 5 : Capture des résultats des tests

8. Machine Learning

8.1 Préparation des données

Le modèle utilise trois variables explicatives : `study_hours`, `attendance_percentage` et `class_participation`. La variable cible est `total_score`. Le grade est dérivé ensuite à partir du score prédit.

8.2 Modèle entraîné

Le modèle principal est un `RandomForestRegressor` intégré dans un pipeline scikit-learn avec `StandardScaler`. Le modèle est sauvegardé dans `models/student_score_model.joblib`.

8.3 Résultats obtenus

Élément	Valeur obtenue
Nombre de lignes	1 000 000
MAE	6.1002
RMSE	8.1978
R2	0.7177
Quality gate	<code>passed_thresholds = true</code>
Modèle enregistré	<code>EduScoreStudentPerformanceModel</code>
Artefacts	<code>student_score_model.joblib</code> , <code>metrics.json</code> , <code>training_reference.json</code> , <code>data_fingerprint.json</code>

Tableau 9 : Résultats du modèle

```
(.venv) PS C:\Users\m\Desktop\Master IA ben msik\S2\MLops\Projet> python -m src.student_score_mlops.train
C:\Users\m\Desktop\Master IA ben msik\S2\MLops\Projet\venv\Lib\site-packages\mlflow\utils\requirements_utils.py:20: UserWarning:
pkg_resources is deprecated as an API. See https://setuptools.pypa.io/en/latest/pkg_resources.html. The pkg_resources package is
 slated for removal as early as 2025-11-30. Refrain from using this package or pin to Setuptools<81.
import pkg_resources # noqa: TID251
Registered model 'EduScoreStudentPerformanceModel' already exists. Creating a new version of this model...
Created version '2' of model 'EduScoreStudentPerformanceModel'.
{
  "mae": 6.10022685166981,
  "rmse": 8.19783018749372,
  "r2": 0.7177169806635926,
  "passed_thresholds": true,
  "dataset_fingerprint": "ee647553f050b82f4b65413d5c289d3ce7703d207b5c9af325280c2e3f32e4db",
  "dataset_row_count": 1000000,
  "training_timestamp": "2026-07-11T11:23:13.859361+00:00",
  "registered_model_name": "EduScoreStudentPerformanceModel",
  "model_uri": "runs/b00d84a864c747119aaf57e06eaa604/model",
  "training_reference": {
    "dataset_fingerprint": "ee647553f050b82f4b65413d5c289d3ce7703d207b5c9af325280c2e3f32e4db",
    "row_count": 1000000,
    "feature_means": {
      "study_hours": 15.029126899999993,
      "attendance_percentage": 84.71104599999998,
      "class_participation": 5.985202599999998
    },
    "created_at": "2026-07-11T11:24:47.080669+00:00"
  }
}
```

Figure 6 : Capture de l'entraînement du modèle

9. Mlflow et MLOps

Mlflow est utilisé pour suivre les expériences. Chaque run enregistre les paramètres du modèle, les métriques, les artefacts, le fingerprint du dataset, le nombre de lignes, le timestamp d'entraînement et le résultat du quality gate.

Catégorie	Informations enregistrées
Paramètres	Type de modèle, n_estimators, max_depth, test_size, random_state.
Métriques	MAE, RMSE, R2, passed_thresholds.
Tags	dataset_fingerprint, training_timestamp, quality_gate.
Artefacts	Modèle, metrics.json, training_reference.json.
Registry	Candidat EduScoreStudentPerformanceModel si les seuils passent.

Tableau 10 : Informations suivies dans Mlflow

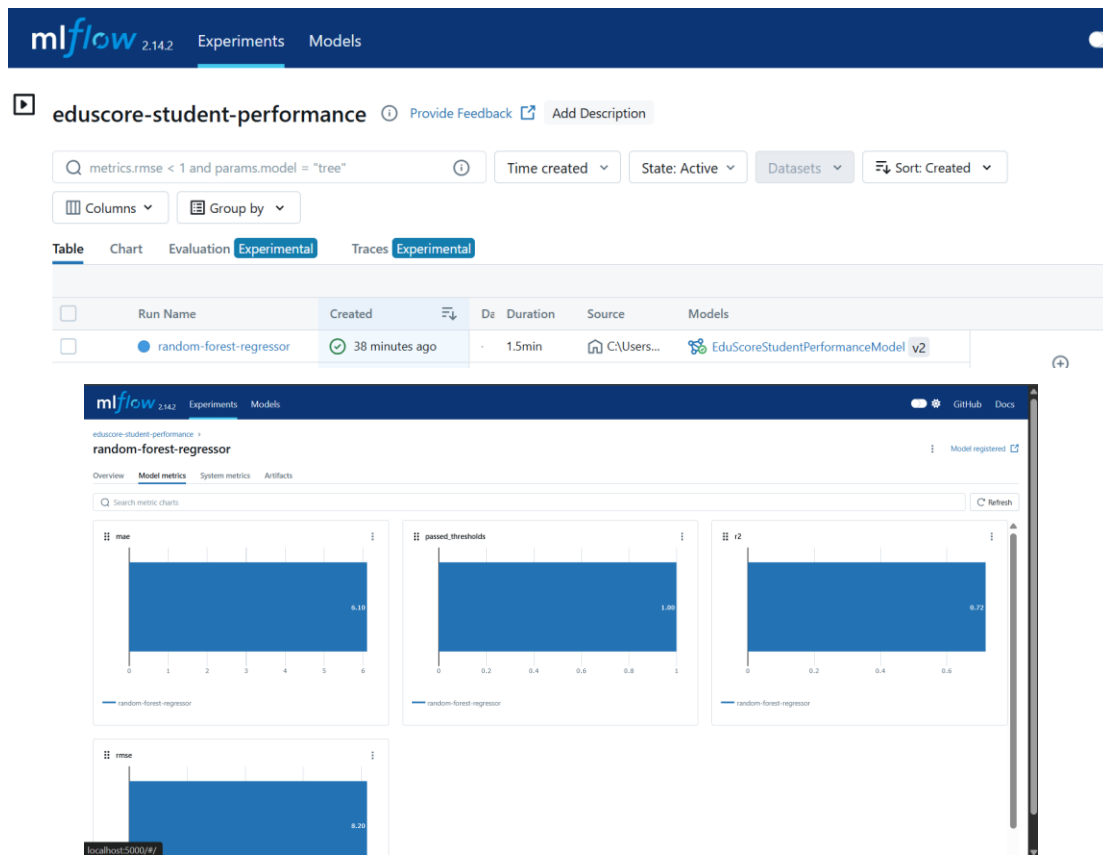


Figure 7 : Capture MLflow des runs

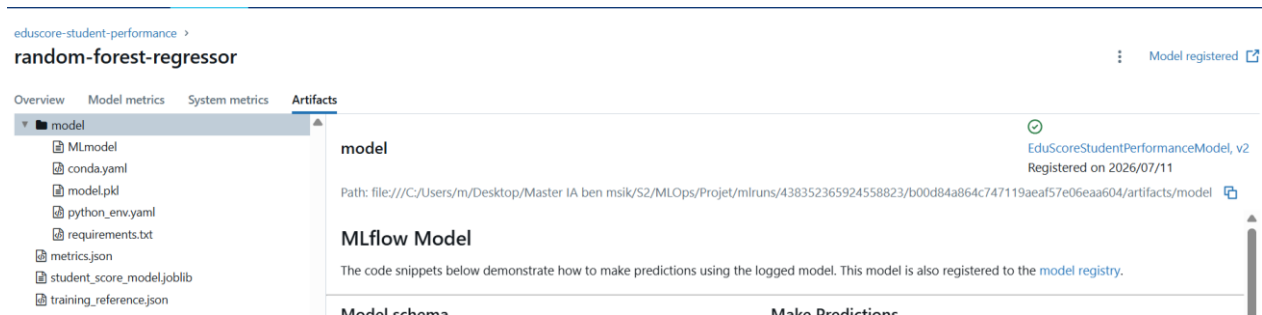


Figure 8 : Capture MLflow du modèle enregistré

10. Déploiement et API

10.1 FastAPI

Méthode	Endpoint	Rôle
GET	/health	Vérifier la disponibilité du service.
GET	/model-info	Afficher le modèle chargé, les métriques et le fingerprint.
POST	/predict	Retourner predicted_score et predicted_grade.

Tableau 11 : Endpoints FastAPI

```

(.venv) PS C:\Users\m\Desktop\Master IA ben msik\S2\MLOps\Projet> uvicorn api.main:app --host 0.0.0.0 --port 8000

INFO:      Started server process [16656]
INFO:      Waiting for application startup.
INFO:      Application startup complete.
INFO:      Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO:      127.0.0.1:57128 - "GET /docs HTTP/1.1" 200 OK
INFO:      127.0.0.1:57128 - "GET /openapi.json HTTP/1.1" 200 OK
INFO:      127.0.0.1:62647 - "GET /model-info HTTP/1.1" 200 OK
INFO:      127.0.0.1:62647 - "GET /favicon.ico HTTP/1.1" 404 Not Found
INFO:      127.0.0.1:49185 - "GET /model-info HTTP/1.1" 200 OK
INFO:      127.0.0.1:49190 - "POST /predict HTTP/1.1" 200 OK
INFO:      127.0.0.1:53224 - "GET /docs HTTP/1.1" 200 OK
INFO:      127.0.0.1:53224 - "GET /openapi.json HTTP/1.1" 200 OK

```

```

• (.venv) PS C:\Users\m\Desktop\Master IA ben msik\S2\MLOps\Projet> Invoke-RestMethod http://localhost:8000/health

status
-----
ok

• (.venv) PS C:\Users\m\Desktop\Master IA ben msik\S2\MLOps\Projet> Invoke-RestMethod http://localhost:8000/model-info

model_path      : C:\Users\m\Desktop\Master IA ben msik\S2\MLOps\Projet\models\student_score_model.joblib
model_exists    : True
loaded_at       : 2026-07-11T11:26:10.005885+00:00
metrics         : @{mae=6,10022685166981; rmse=8,19783018749372; r2=0,7177169806635926; passed_thresholds=True;
dataset_fingerprint=ee647553f050b82f4b65413d5c289d3ce7703d207b5c9af325280c2e3f32e4db;
dataset_row_count=1000000; training_timestamp=2026-07-11T11:23:13.859361+00:00;
registered_model_name=EduScoreStudentPerformanceModel;
model_uri=runs:/b00d84a864c747119aeaf57e06eaa604/model; training_reference=}
dataset_fingerprint : @{fingerprint=ee647553f050b82f4b65413d5c289d3ce7703d207b5c9af325280c2e3f32e4db; row_count=1000000;
source=duckdb:ml_student_training_set; updated_at=2026-07-11T11:24:47.102750+00:00}
reported_at      : 2026-07-11T12:04:23.996478+00:00

```

10.2 Docker

Docker permet de lancer l'API et MLflow dans un environnement reproductible. Le modèle doit être entraîné localement avant le lancement du conteneur API afin que `models/student_score_model.joblib` soit disponible.

11. CI/CD et collaboration technique

Le workflow GitHub Actions automatise les vérifications principales: installation des dépendances, lint avec Ruff, tests avec pytest et build Docker. Cette chaîne garantit un niveau minimal de qualité avant intégration.

Étape	Objectif
Install dependencies	Installer Python et requirements.txt.
ruff check .	Contrôler la qualité syntaxique et style.
pytest -q	Exécuter les tests Python.
docker build	Vérifier que l'image Docker se construit.

Tableau 12 : Pipeline CI/CD GitHub Actions

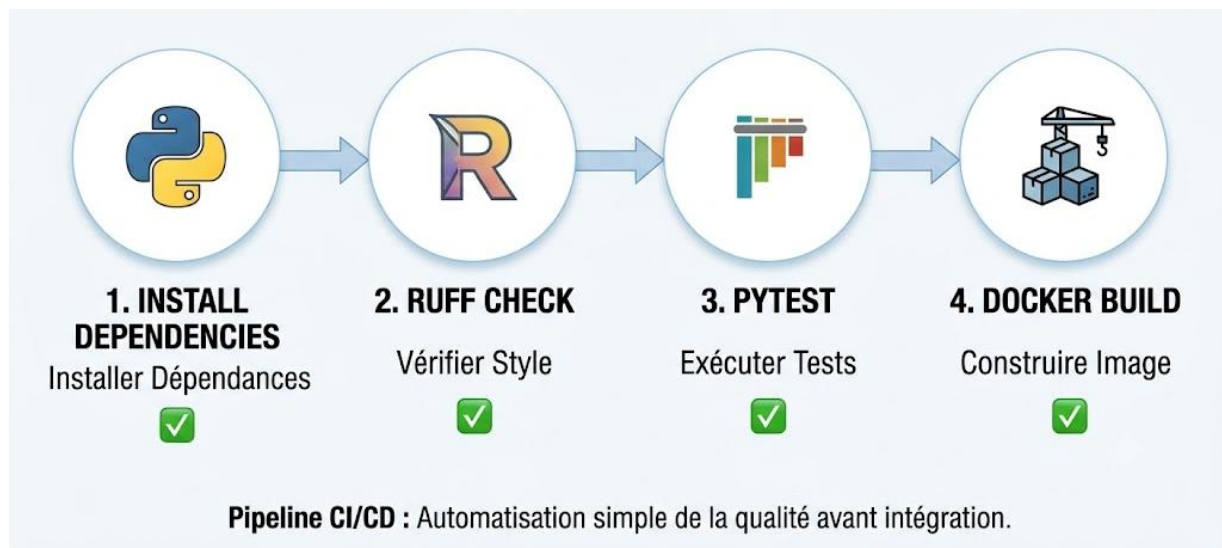


Figure 9 : Pipeline CI/CD

12. Monitoring et observabilité

Chaque prédiction est journalisée dans `monitoring/predictions.jsonl`. Le log contient un timestamp, les variables d'entrée, le score prédit, le grade prédit et la latence en millisecondes. L'endpoint `/model-info` complète l'observabilité en exposant les informations du modèle chargé.

Information	Utilité
timestamp	Tracer le moment de la prédiction.
features	Conserver les variables utilisées.
predicted_score	Analyser les sorties numériques.
predicted_grade	Interpréter la sortie métier.
latency_ms	Surveiller le temps de réponse.
dataset_fingerprint	Relier le modèle à la version du dataset.

Tableau 13 : Données observées en monitoring

13. Validation globale et résultats de démonstration

La validation complète consiste à restaurer la donnée, construire le pipeline, exécuter les tests, entraîner le modèle, lancer MLflow, lancer l'API, effectuer une prédiction et vérifier les logs.

Étape	Commande ou preuve	Résultat attendu
DVC	scripts\dvc.cmd pull / status	Dataset disponible et synchronisé.
Ingestion	python dlt_pipeline/load_student_data.py	DuckDB mis à jour.
dbt	dbt run puis dbt test	Modèles et tests validés.
Tests Python	pytest -q	12 passed.
Training	python -m src.student_score_mlops.train	Modèle généré, quality gate validé.
MLflow	mlflow ui --backend-store-uri mlruns	Runs, métriques et artefacts visibles.
API	uvicorn api.main:app --port 8000	Service disponible.
Prediction	POST /predict	Score et grade retournés.
Monitoring	Get-Content monitoring/predictions.jsonl	Log de prédiction créé.

Tableau 14 : Checklist de validation finale

14. Synthèse des livrables réalisés

Exigence	Réalisation
Vision projet	Problématique, objectifs, utilisateurs, Data Strategy.
Agile	Jira, epics, stories, sprints, répartition des tâches.
GitHub	Dépôt, README, documentation, workflow CI/CD.
DataOps	DVC, dlt, DuckDB, dbt, Dagster.
Qualité	Data contract, dbt tests, pytest, Pydantic.
Machine Learning	Préparation, entraînement, évaluation, sauvegarde.
MLflow	Tracking, métriques, artefacts, registry.
Déploiement	FastAPI et Docker.
Monitoring	Logs prédiction, latence, model-info, fingerprint.
Documentation	README, docs techniques et rapport final.

Tableau 15 : Correspondance entre exigences du module et réalisations

Conclusion

Le projet EduScore MLOps démontre la mise en œuvre complète d'une solution Data/IA selon les principes du DataOps, du MLOps et de l'Agilité. Il montre comment un dataset brut peut être transformé progressivement en un service de prédiction exploitable, reproductible et observable. La valeur du projet ne réside donc pas uniquement dans le modèle de régression, mais dans toute la chaîne qui permet de produire, valider, suivre, déployer et monitorer ce modèle.

La solution réalisée couvre les principales exigences du module: ingestion des données, stockage local, transformations, tests qualité, data contract, orchestration, entraînement Machine Learning, suivi des expériences, registry MLflow, API FastAPI, conteneurisation Docker, CI/CD GitHub Actions, monitoring, documentation et collaboration GitHub/Jira. Les résultats obtenus montrent un modèle entraîné sur un dataset d'un million de lignes, avec un quality gate validé, des métriques suivies dans MLflow et une API capable de servir des prédictions.

Ce projet constitue ainsi une base solide pour une solution de Learning Analytics plus avancée. Dans une évolution future, il serait possible d'ajouter un tableau de bord de monitoring, une analyse d'explicabilité du modèle, une stratégie de promotion automatique des modèles et une intégration avec une plateforme pédagogique comme Moodle.

Bibliographie

1. Dépôt GitHub du projet: <https://github.com/Bahriilhame/mlops-project>
2. Student Performance Dataset, Kaggle.
3. Documentations officielles: DVC, dlt, DuckDB, dbt, Dagster, scikit-learn, MLflow, FastAPI, Docker, GitHub Actions.
4. Kreuzberger, D., Kühn, N., Hirschl, S. Machine Learning Operations (MLOps): Overview, Definition, and Architecture, 2022.

Annexe A: Commandes complètes d'exécution

```
cd "C:\Users\m\Desktop\Master IA ben msik\S2\MLOps\Projet"

.\.venv\Scripts\Activate.ps1

pip install -r requirements.txt

scripts\dvc.cmd pull

python dlt_pipeline/load_student_data.py

dbt run --project-dir dbt/student_score --profiles-dir dbt/student_score

dbt test --project-dir dbt/student_score --profiles-dir dbt/student_score

pytest -q

python -m src.student_score_mlops.train

mlflow ui --backend-store-uri mlruns --host 0.0.0.0 --port 5000

uvicorn api.main:app --host 0.0.0.0 --port 8000

docker compose up --build
```

Annexe B: Exemple de requête de prédiction

```
$body = @{
    study_hours = 12
    attendance_percentage = 88
    class_participation = 7
} | ConvertTo-Json

Invoke-RestMethod -Uri http://localhost:8000/predict -Method POST -ContentType
"application/json" -Body $body
```